

Computer Architecture Study

Master:

Dr. Safari

Project:

multi-Cycle Design for RISC-V ISA

Contributors:

Anita Zekavat & Amir Hosseini

Project Workflow

1. RTL Component Identification:

Initially, we performed a comprehensive analysis of the instruction set architecture (ISA) to identify the necessary Register Transfer Level (RTL) components, such as the ALU, Register File, Program Counter (PC), and Memory units required for a multi-cycle implementation.

2. Control Unit Design:

We then focused on designing the Control Unit using a Finite State Machine (FSM) approach. This was a crucial step to determine the precise control signals needed for each execution stage (Fetch, Decode, Execute, Memory, and Write-back).

3. Datapath Development:

Based on the control logic requirements, we architected the Datapath. Designing the datapath after the controller allowed us to have a clear blueprint of the necessary interconnections and multiplexer selections needed to route data correctly through the processor.

4. Top Module Integration:

Finally, we integrated the Datapath and the Control Unit into a Top-Level Module. This involved interconnecting all sub-modules and ensuring that the synchronization between the control signals and data flow was perfectly established for hardware simulation.

system Hierarchy and Integration:

The overall system design is encapsulated within a Top-Level Module, which integrates two primary independent units: the CPU and the Memory. The detailed structural design of this integration is discussed in below Section :

`"$PATH/TO/PROJECT"/design/papers/Top_paper`

The CPU module itself is subdivided into two main architectural components: the Datapath and the Control Unit (CU). The implementation details and schematic diagrams for these units can be found in Sections

`"$PATH/TO/PROJECT"/design/papers/datapath_paper`

and `"$PATH/TO/PROJECT"/design/papers/CU_paper`

, respectively.

Control Unit Detailed Design

We will now focus on the internal architecture of the Control Unit. In our design, the CU is split into two functional parts:

1. ALU/Functional Controller:

For the functional control logic, we have defined a specific truth table to map instruction fields to ALU operations. This mapping is illustrated in the table below:

”\$PATH/TO/PROJECT”/design/tabels/
Func_Controller_table

2. Main Controller Detailed Architecture:

”\$PATH/TO/PROJECT”/design/papers/maincontroller_table

Based on the architectural schematic, the Main Controller follows a sophisticated two-level microprogrammed structure. The components and their interconnections are defined as follows:

1. Micro-PC:

A 5-bit register that holds the current state address cnt_PC. It is updated every clock cycle with the next_pc value generated by the sequencing logic.

2. Logic:

This combinational block is the brain of the sequencer. It

determines the next_pc by evaluating three inputs: the 7-bit Opcode from the instruction, the current state (cnt_pc), and a 2-bit Next-state control signal fed back from the Micro-memory.

3. Micro-memory:

Indexed by the cnt_pc this memory unit serves two purposes:

It provides a 4-bit *Index to address the Nano-memory.

It outputs a 2-bit *Next signal to the Logic block to assist in determining the next micro-instruction address.

4. Nano-memory:

This unit acts as the final control store. It receives the 4-bit index from the Micro-memory and decodes it into the final set of control signals, including RegWrite, ALUSrcA/B, ALUOp, MemWrite, and others.

By using this Micro-Nano architecture, we significantly reduced the redundancy in the control store, as multiple micro-states that share the same control signals can point to the same entry in the Nano-memor

In the Verilog implementation, instead of using standard ROM structures, we utilized Programmable Array Logic (PAL)

concepts for the two memory units. By explicitly defining the memory size and bit-width within the Verilog code, we ensured an optimized and synthesis-friendly hardware realization of the control store.